

Fit5225 A1 作业记录

为什么我可以直接这样调用 `python script`，然后为什么我可以直接 `client/inputfolder` 后面接 `api` 的 `url`，这参数是怎么传进去的？

代码块

```
1 python client/cloudpose_client.py client/inputfolder
  http://localhost:60001/api/pose_estimation 1
```

其实就是：运行一个 Python 文件 + 给它传 3 个命令行参数。

1) `python client/cloudpose_client.py` 到底做了什么？

- `python`：调用当前环境里的 Python 解释器（你现在是 `.venv` 里那个）
- `client/cloudpose_client.py`：告诉解释器“执行这个文件”

等价于你对 Python 说：

“把这个 `.py` 文件当作程序入口跑起来。”

后面的：`client/inputfolder`；http://localhost:60001/api/pose_estimation；`1` 叫做 **命令行参数 (command-line arguments)**。

Python 运行脚本时，会把这些参数原封不动塞进一个列表里：**`sys.argv`**

`sys.argv` 到底长啥样？

代码块

```
1  [
2      "client/cloudpose_client.py",           # sys.argv[0] 永远是脚本名
3      "client/inputfolder",                   # sys.argv[1] 第一个参数
4      "http://localhost:60001/api/pose_estimation", # sys.argv[2] 第二个参数
5      "1"                                     # sys.argv[3] 第三个参数（注意：是字
      符串）
6  ]
```

为什么可以 “inputfolder 后面接 URL” ？

在命令后面写的东西，都按空格分隔，依次变成参数。

关键点是：

这些参数的“含义”不是命令行决定的，而是脚本作者自己在代码里规定的。

你的脚本规定了：

- 第 1 个参数：图片目录
- 第 2 个参数：API URL
- 第 3 个参数：并发线程数

所以你就必须按这个顺序传。如果你换顺序，它就会读错。

此处对应 `2025_FIT5225_A1/client/cloudpose_client.py` 中的 main.py 文件

代码块

```
1  def main():
2      ## provide argumetns-> input folder, url, number of wrokers
3      if len(sys.argv) != 4:
4          raise ValueError("Arguments list is wrong. Please use the following
format: {} {} {} {}".format("python Cloudpose_client.py", "<input_folder>", "<URL>", "<number_of_workers>"))
5
6
7      input_folder = os.path.join(sys.argv[1], "")
8      images = get_images_to_be_processed(input_folder)
9      num_images = images.__len__()
10     num_workers = int(sys.argv[3])
11     start_time = time.time()
12     #craete a worker thread to invoke the requests in parallel
13     with PoolExecutor(max_workers=num_workers) as executor:
14         for _ in executor.map(call_cloudpose_service, images):
15             pass
16     elapsed_time = time.time() - start_time
17     print("Total time spent: {} average response time:
{}".format(elapsed_time, elapsed_time/num_images))
18
```

进阶refactor方法：使用argparse（工程级别）

如你所见，上面的调用方式是最粗糙的调用，需要人工记住每个参数的顺序，如果命令行输错则会直接报错。

工程级写法应该是：

代码块

```
1 python3 ./client/cloudpose_client.py \  
2     --input-folder client/inputfolder \  
3     --url http://localhost:60001/api/pose_estimation \  
4     --workers 4
```

**** 看完整的 argparse 结构，点击这个连接：**
https://github.com/Oatsinthepine/25_fit5225_A1/blob/main/client/cloudpose_client.py

最终架构，整理后我们有：

层	职责
client	压测工具
app/ main.py	HTTP 接口
app/ pose_detection.py	推理逻辑
app/model	模型文件

这是一个标准的：

单体 AI 微服务结构

为什么不先 Docker 化，而是先做负载测试？

一句话核心👉 先验证性能，再容器化。

因为：

如果裸机都跑不稳，Docker 只会把问题打包带走。

为什么必须先在本地图压测？

因为现在：

- 没有 Docker 层
- 没有 K8s 层
- 没有网络开销
- 没有容器资源限制

这时的性能是：

纯模型 + Python + FastAPI 的真实性能

这叫：baseline（基准）

真正的云工程流程是：

- 1 本地功能正确
- 2 本地性能 baseline
- 3 容器化
- 4 容器内再次压测
- 5 K8s 部署
- 6 集群压测

这是完整工业流程。

Baseline 并发测试（测试的是detect_base64_img()）：

- base64 decode
- YOLO 推理
- JSON 组装

◆ 当前 cloudpose_client 是什么？

它是：

- 本地多线程
- 固定图片数量

- 批量并发
- 不是持续压力

它适合：

初步观察并发影响

◆ Locust 是什么？

Locust 是：

- 持续流量生成
- 模拟用户行为
- 支持 ramp-up
- 支持实时统计
- 可视化 dashboard

它适合：

系统级压测

代码块

```
1 python3 ./client/cloudpose_client.py \  
2 --input-folder client/inputfolder \  
3 --url http://localhost:60001/api/pose_estimation \  
4 --workers 1/5/10/20
```

Workers	Total Time	Avg Response Time
1	14.43	0.1127
5	14.06	0.1098
10	14.01	0.1094
20	14.08	0.11

⚠ 重点：几乎完全一样。

第一层分析：这说明什么？

这说明：

你的服务是完全 CPU-bound 且推理是串行的。

为什么？

因为：

- 128 张图片
- 无论 1 worker 还是 20 worker
- 总耗时始终 ≈ 14 秒

这意味着：

系统吞吐量是固定的

并发没有提升吞吐

第二层分析：为什么没有变慢？

我们之前预期：

多 worker 会因为 lock 导致排队，平均时间上升

但你没看到。

为什么？

因为：

- 128 张图片总量固定
- 模型推理时间固定
- ThreadPoolExecutor 只是把请求发得更快
- 但真正推理是串行执行

换句话说：

你不是在“压系统”

你只是“排队完成 128 次任务”

现在这组数据的工程意义

报告里写：

Increasing client-side concurrency (1–20 workers) did not significantly change total execution time, indicating that the service is CPU-bound and inference is serialized.

最关键总结

你在这组数据说明：

你的系统已经达到 CPU 极限

并发没有带来性能提升

系统是 compute-bound 而不是 IO-bound

这其实是非常典型的 AI 推理服务特征

下面是 fastapi worker=1 时的 Locust 压测结果

代码块

```
1 uvicorn cloudpose.app.main:app --host 0.0.0.0 --port 60001 --workers 1
```

STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS

STATISTICS CHARTS FAILURES EXCEPTIONS CURRENT RATIO DOWNLOAD DATA LOGS												
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/poee_estimation	2036	0	240	430	520	253.93	66	738	2224.48	9	0
	Aggregated	2036	0	240	430	520	253.93	66	738	2224.48	9	0

STATISTICS												CHARTS	FAILURES	EXCEPTIONS	CURRENT RATIO	DOWNLOAD DATA	LOGS
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s					
POST	/api/poee_estimation	166	0	620	1200	1600	605.31	98	1665	1977.78	8.9	0					
	Aggregated	166	0	620	1200	1600	605.31	98	1665	1977.78	8.9	0					

STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS

<

这是一个教科书级 CPU-bound 系统曲线 🔥

我们把四组数据放在一起看：

Users	Avg Latency	95%	QPS
1	134 ms	170 ms	2.6
5	254 ms	430 ms	9
10	605 ms	1200 ms	8.9
20	1199 ms	3000 ms	9

最关键现象

◆ QPS（query per second）

从 user=5 开始：

QPS ≈ 9
再也没变过

说明：

- 👉 你的机器最大吞吐 ≈ 9 req/s
- 👉 系统已经达到计算极限

◆ Latency

User 增加：

- 5 → 10 → 20

- 延迟几乎线性上涨

这说明：

👉 请求在排队

👉 系统饱和

延迟增长趋势

Users	Avg Latency
5	254 ms
10	605 ms
20	1199 ms

几乎翻倍。

这正是：

单线程串行 + CPU-bound + 队列堆积

这说明 lock 是真正瓶颈

代码块

```
1  with lock:
2      results = model(decoded_img)
```

即使：

- 20 users
- 20 并发请求

模型仍然一次只执行 1 个。

所以：请求队列长度 $\approx$ 用户数。

为什么 QPS 不下降？因为：

系统在满负载运行。每秒最多完成 9 次推理。无论多少用户：
完成速度不变。
只是排队时间增加。

这就是典型的 M/M/1 队列模型

你现在的系统行为符合：

单服务节点
单服务器
排队系统

这在云计算里非常经典。

现在我们思考扩展策略：

现在有三种扩展方式：

1 提高单实例并行度

理论上：

- 去掉 lock
- 用多进程
- 或 uvicorn 多 worker

但 YOLO 本身未必线程安全。

2 垂直扩展（Scale Up）

- 更强 CPU
 - GPU 推理
-

3 水平扩展 (Scale Out)

- 多副本
- Kubernetes Deployment
- LoadBalancer 分流

这才是云原生做法。

现在你理解 Kubernetes 的意义了吗？

因为单实例：

- 吞吐固定
- 延迟随用户线性上升

但如果你有：2 个副本：

吞吐 $\approx 2 \times 9 = 18$ req/s

延迟降低

这就是：Horizontal scaling

下面进行单机多进程扩展 (Vertical parallelism) 并重新进行压测。

注意这次我们 worker = 4，上面是 1

```
1  uvicorn cloudpose.app.main:app --host 0.0.0.0 --port 60001 --workers 4
```

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/poise_estimation	46	0	130	170	220	135.09	88	220	1770.26	2.2	0
Aggregated		46	0	130	170	220	135.09	88	220	1770.26	2.2	0

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/poise_estimation	390	0	180	300	370	189.67	85	380	2205.07	10.5	0
Aggregated		390	0	180	300	370	189.67	85	380	2205.07	10.5	0

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/poise_estimation	502	0	300	860	1200	403.3	114	1455	2030.95	13.4	0
Aggregated		502	0	300	860	1200	403.3	114	1455	2030.95	13.4	0

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/poise_estimation	579	0	750	1900	1900	736.34	109	2149	2195.44	16.1	0
Aggregated		579	0	750	1900	1900	736.34	109	2149	2195.44	16.1	0

这是单进程 vs 多进程 的本质区别。

先整理你这轮数据（uvicorn -workers 4）

Users	Avg Latency	QPS
1	135 ms	2.2
5	189 ms	10.5
10	403 ms	13.4
20	736 ms	16.1

对比之前（workers=1）：最大 QPS ≈ 9

现在：最大 QPS ≈ 16

👉 吞吐几乎翻倍。

worker=4 是✅是 4 个独立进程（process）。

实际发生的是：

- 启动 4 个独立 Python 进程
- 每个进程：
 - 有自己的 FastAPI 实例
 - 有自己的内存空间
 - 有自己的 YOLO 模型实例
 - 有自己的 lock

它们是完全独立的。

现在是 4 路并行推理

- 1 4 进程
- 2 20 用户
- 3 → 分摊到 4 个进程
- 4 → 每个进程只排 5 个

为什么不是 4 倍？

理论最大： $9 \text{ req/s} \times 4 \approx 36 \text{ req/s}$

但你只到 16 req/s。

因为：

- CPU 核心数量有限
- 进程调度开销
- 模型初始化重复占内存
- base64 编码解码开销
- FastAPI 框架开销

所以不会线性 4x。

之前的 lock 只锁 **进程内**。

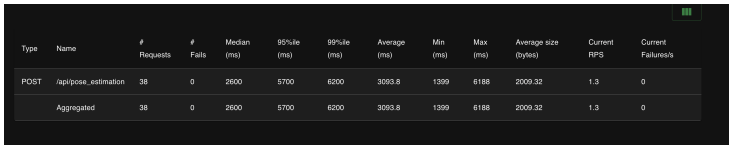
不同进程之间：

- ✗ 不共享 lock
- ✗ 不共享模型
- ✗ 不共享内存

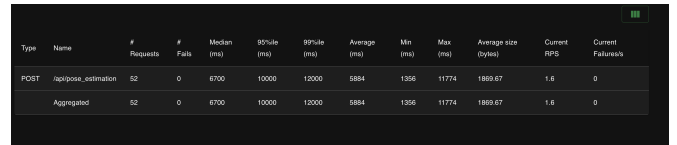
在 Docker 里进行对于 fastapi 服务的 Locust 压测：

代码块

```
1 locust -f client/locustfile.py --host=http://localhost:60001
2
3 # 在另一个terminal 运行
4 docker run -p 60001:60001 cloudpose:latest --worker=2
```



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	api/pose_estimation	38	0	2800	5700	6200	3593.8	1399	6188	2009.32	1.3	0
	Aggregated	38	0	2800	5700	6200	3593.8	1399	6188	2009.32	1.3	0



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	api/pose_estimation	52	0	6700	10000	12000	8884	1356	11774	1869.67	1.6	0
	Aggregated	52	0	6700	10000	12000	8884	1356	11774	1869.67	1.6	0

为什么 Docker 容器里的 QPS 相比裸机低了很多：

Docker Desktop 实际运行在：

| 一个 Linux 虚拟机里

不是直接在 macOS 上。

代码块

```
1 Mac
2   └─ Docker Desktop VM
3       └─ Linux
4           └─ 你的容器
```

你现在是：

| Mac → VM → 容器 → Python → YOLO

多了一层虚拟化。

第4部分：K8s cluster + K8s services

Step 1：把 Docker 镜像 push 到 DockerHub

但是先得给镜像打Tag

代码块

```
1 docker tag cloudpose <你的dockerhub用户名>/cloudpose:v1
```

然后记得在terminal `docker login`

之后再push

代码块

```
1 docker push coolguyinurarea/cloudpose:v1
```

🚧 Assignment 1- 容器化 & 云部署踩坑总结

💣 坑 1: MacOS 本地打镜像 vs 云端操作系统架构不兼容

K8s Pod 启动失败，日志显示：

代码块

```
1 exec /usr/local/bin/uvicorn: exec format error
2
3 # Pod 状态
4 Exit code 255
```

🔍 根本原因

- 本地开发机器：MacBook (Apple Silicon, ARM64)
- OCI K8s 节点：Linux x86_64 (AMD64)

默认执行了 `docker build` 直接构建出的镜像架构为：linux/arm64

当 OCI (x86_64) 拉取 ARM 镜像时：

→ Linux 内核拒绝执行二进制

→ 报错 exec format error

→ 容器直接退出

解决方案：

使用 docker build 强制构建 arm64 镜像：

代码块

```
1 docker buildx create --use
2 docker buildx inspect --bootstrap
```

```
3
4 docker buildx build \
5     --platform linux/amd64 \
6     -t <your-image>:v2 \
7     --push .
8
9     # 关键点:
10    -- platform linux/amd64
```

- Apple Silicon 本地开发必须显示指定平台
- Docker 镜像包含 CPU 架构
- ARM 镜像不能在 x86 机器上运行

以后看到 exec format error

第一时间想到 镜像与 instance **架构不匹配**

 **坑 2:**  **大镜像 + 超长上传时间 + DockerHub 连接超时导致失败的问题:**

问题现象 docker buildx --push 长时间卡在 pushing layers，最终报错

代码块

```
1 write: broken pipe
2 use of closed network connection
3 failed to push blob
```

根本原因

镜像体积过大

- 上传时间超过 30~60 分钟
- DockerHub 或中间网络设备关闭 TCP 连接
- Push 失败

解决方案一（临时）

直接重试：

```
docker buildx build --platform linux/amd64 --push .
```

有时第二次会成功。

✅ 解决方案二（推荐）

添加 `.dockerignore` 减小 build context：

```
.dockerignore
```

```
1 client/  
2 scripts/  
3 *.jpg  
4 *.csv  
5 pycache/  
6 .git
```

避免把无关文件传入构建环境。

✅ 解决方案三（最稳）

直接在 OCI x86 机器上 build：

代码块

```
1 ssh ubuntu@k8s-master  
2 docker build -t <your-image>:v2 .  
3 docker push <your-image>:v2
```

优点：

- 不需要跨架构构建
 - 网络更稳定
 - 推送成功率高
-



经验总结

- AI 推理镜像 $\geq 2\text{GB}$ 很常见
- 家庭网络 push 大镜像极易失败
- 云服务器 build + push 更稳定

🔪 broken pipe $\neq$ Docker bug

通常是大镜像 + 长时间上传

🔪 坑 3: K8s Deployment 端口不一致

🧠 问题现象

容器监听:

60001

Service targetPort:

60000

导致 NodePort 无法访问。

🔍 原因

K8s Service 只会转发到 `targetPort` 指定端口。

端口必须一致。

✅ 解决方案

统一:

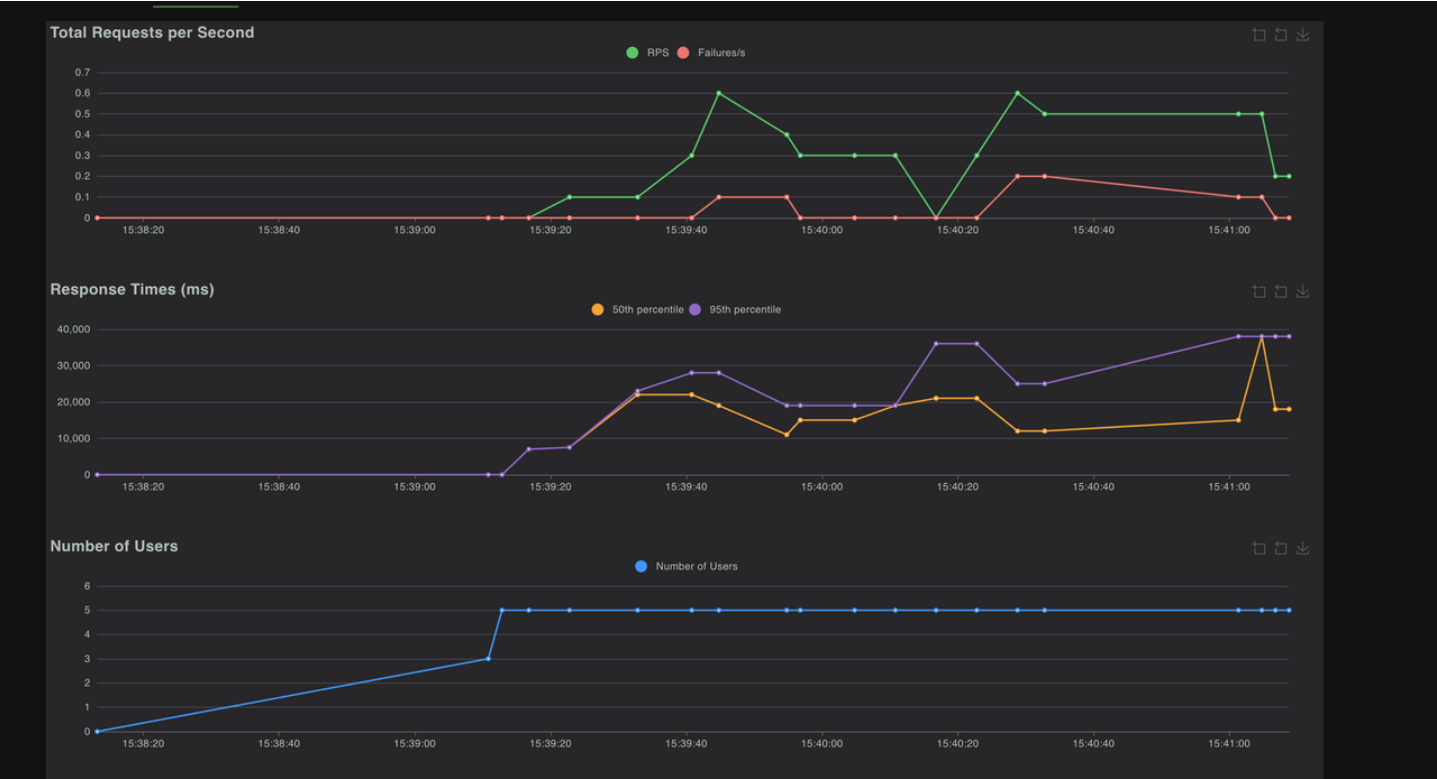
- Dockerfile
- Unicorn port
- containerPort
- Service targetPort

A1 第 6、第 7 部分: Locust压测与报告:

以下都是从本地发起的Locust压测请求给OCI k8s 集群（1 master 2 workers）

Replica = 1 ; user = 5 时：

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/pose_estimation	33	4	15000	36000	38000	16283.86	3195	38182	1106	0.2	0
	Aggregated	33	4	15000	36000	38000	16283.86	3195	38182	1106	0.2	0



当前实验条件

- replicas = 1
- memory = 1Gi
- users = 5
- duration = 2 mins
- 外网 NodePort 访问

结果核心数据

请求统计

- Requests: 33
 - Fails: 4
 - Avg response time: **16283 ms (16.3秒)**
 - Median: 15000 ms
 - 95%: 36000 ms
 - Max: 38182 ms
 - RPS: 0.2
-

第一层分析：延迟

16 秒平均延迟说明什么？

这不是网络问题。

这是：🔥 模型推理时间 -- **每个请求几乎是串行执行的。**

第二层分析：RPS

0.2 RPS

意味着：

每秒 0.2 请求

也就是 1 个请求需要约 5 秒

但平均响应 16 秒，说明：

- **请求是排队的**
 - **CPU 是瓶颈**
 - 单 worker 模式
-

第三层分析：失败

4/33 失败 $\approx 12\%$

这已经很高了。

原因可能是：

- 请求超时
- pod 资源临界
- uvicorn 默认 timeout
- Locust 默认 timeout

结合RPS 图

峰值 0.6 RPS

说明偶尔有短时间并发成功。

但整体维持低吞吐。

Response Time 图

95% 接近 36 秒

说明：

系统开始积压请求

这非常典型。

结论（第一轮实验）

你现在的系统是：

- CPU-bound
- 单实例
- 单 worker
- 无法并发推理

- 请求串行排队

我们要验证一件事：

是 CPU 瓶颈还是 Pod 数量瓶颈？

如果系统是 CPU-bound：

- 延迟应该下降
- RPS 应该翻倍
- 失败率下降

如果没有明显变化：

说明模型加载方式有问题。

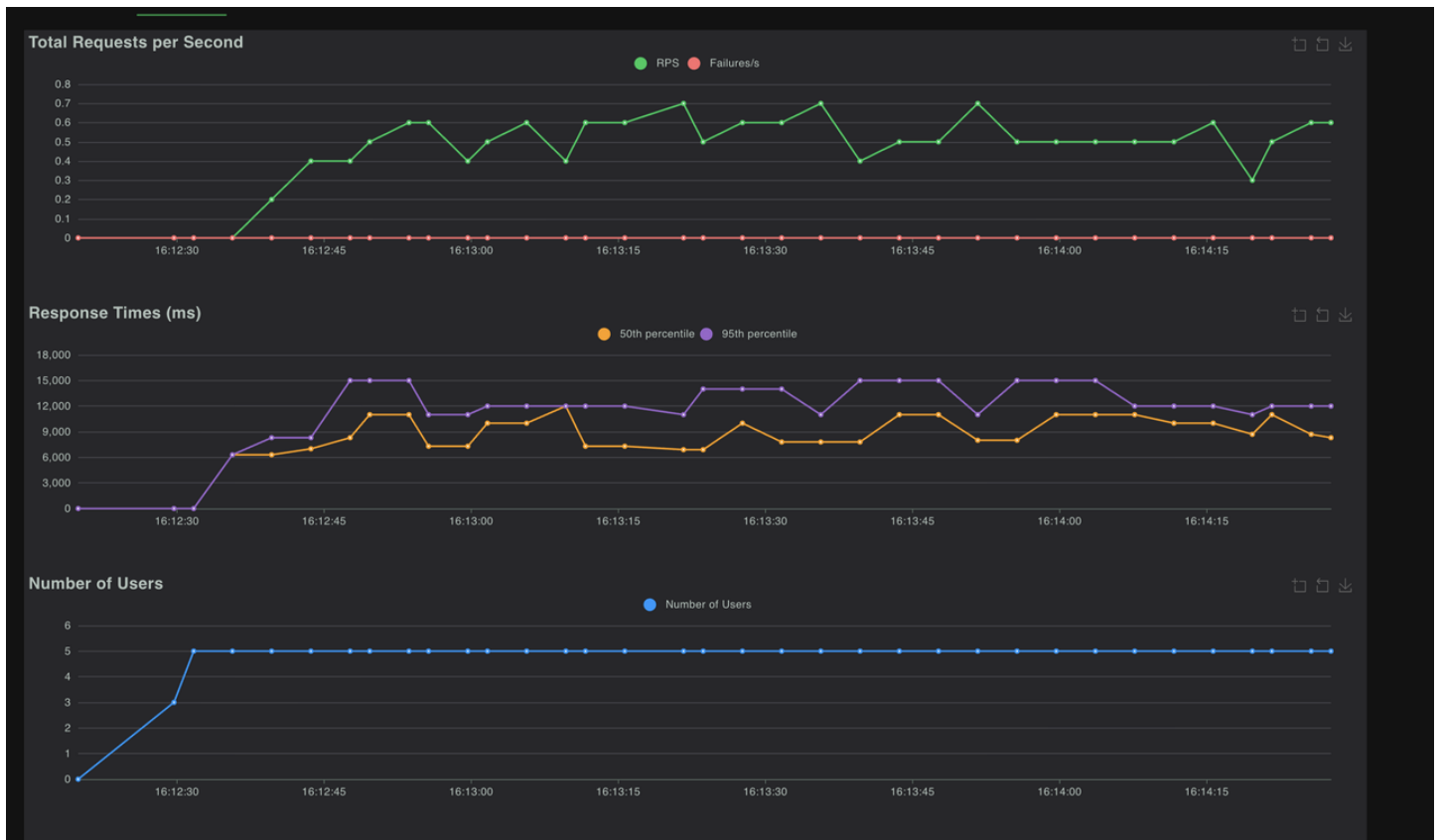
关键点

横向扩展是否有效？

这就是 Kubernetes 核心。

Replica = 2； user = 5 时：

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/pose_estimation	60	0	8000	15000	15000	9058.73	5783	15190	2651.72	0.6	0
	Aggregated	60	0	8000	15000	15000	9058.73	5783	15190	2651.72	0.6	0



对比：Replica = 1 vs Replica = 2

◆ 单副本（之前）

- Requests: 33
- Fails: 4
- Avg latency: **16283 ms**
- 95%: 36000 ms
- RPS: ~0.2
- 失败率：12%

◆ 双副本（现在）

- Requests: 60
- Fails: 0
- Avg latency: **9058 ms**
- 95%: 15000 ms

- RPS: ~0.6
- 失败率: 0%

第一层结论：吞吐量

RPS 从: 0.2 → 0.6

几乎翻了 3 倍。

这说明：

- Service 在做负载均衡
- kube-proxy 正常工作
- 两个 Pod 都在处理请求

横向扩展有效。

第二层结论：延迟

平均延迟: 16s → 9s

降低约 44%。

95% 延迟:

| 36s → 15s

降低超过 50%。

说明:

| 请求不再严重排队

单副本时 CPU 被打满，请求堆积。

双副本时请求被分摊。

第三层结论：稳定性

失败从：4 → 0

说明：

- 系统不再过载
 - 不再触发 timeout
 - 不再临界资源波动
-

这说明什么？

你的服务是：

| CPU-bound + 推理串行执行

当只有一个副本时：

- 所有请求排队
- latency 爆炸
- 出现失败

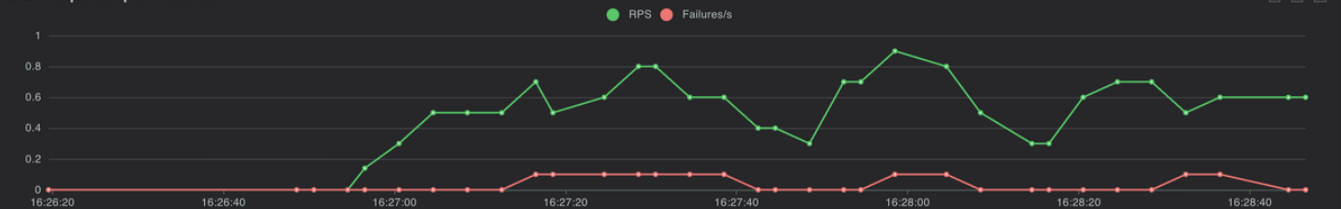
当增加副本：

- 负载被均分
 - 延迟下降
 - 失败消失
-

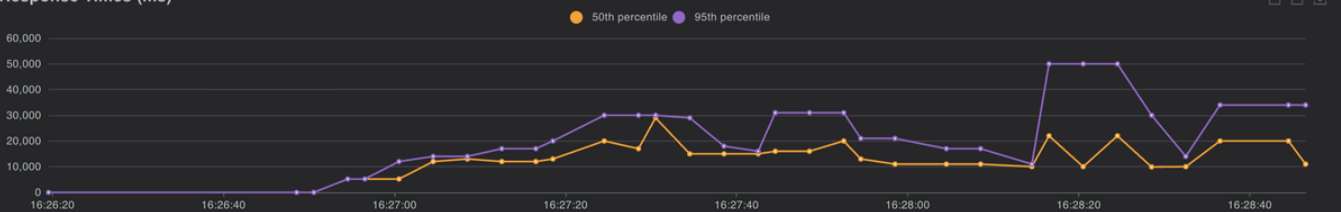
Replica = 2； user = 10 时：

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/pose_estimation	63	4	13000	31000	50000	15364.08	3267	49687	2289.9	0.6	0
	Aggregated	63	4	13000	31000	50000	15364.08	3267	49687	2289.9	0.6	0

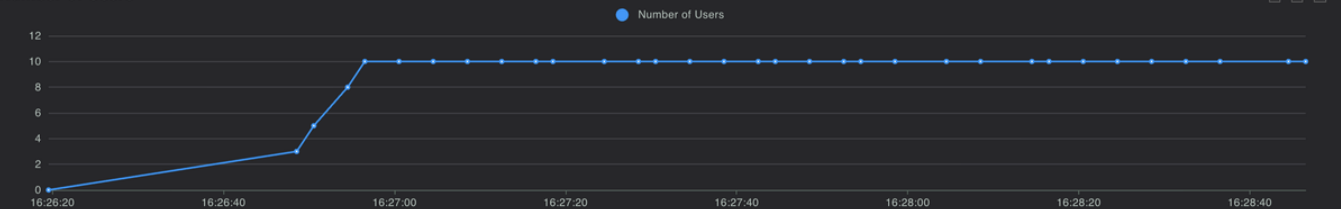
Total Requests per Second



Response Times (ms)



Number of Users



当前结果 (replica = 2, users = 10)

指标

- Requests: 63
- Fails: 4
- Avg latency: **15.3s**
- 95%: 31s
- 99%: 50s
- RPS: 0.6

这说明什么?

当 users 从 5 → 10:

- RPS 没有翻倍
- 延迟明显上升

- 失败又出现

这说明：

| 两个 Pod 已经被打满

你的系统瓶颈现在是：

- CPU
- 或推理时间串行阻塞

不是 Service，不是网络。

关键观察

Replica = 2 时

Users = 5 $\rightarrow$ RPS $\approx$ 0.6

Users = 10 $\rightarrow$ RPS 还是 $\approx$ 0.6

说明：

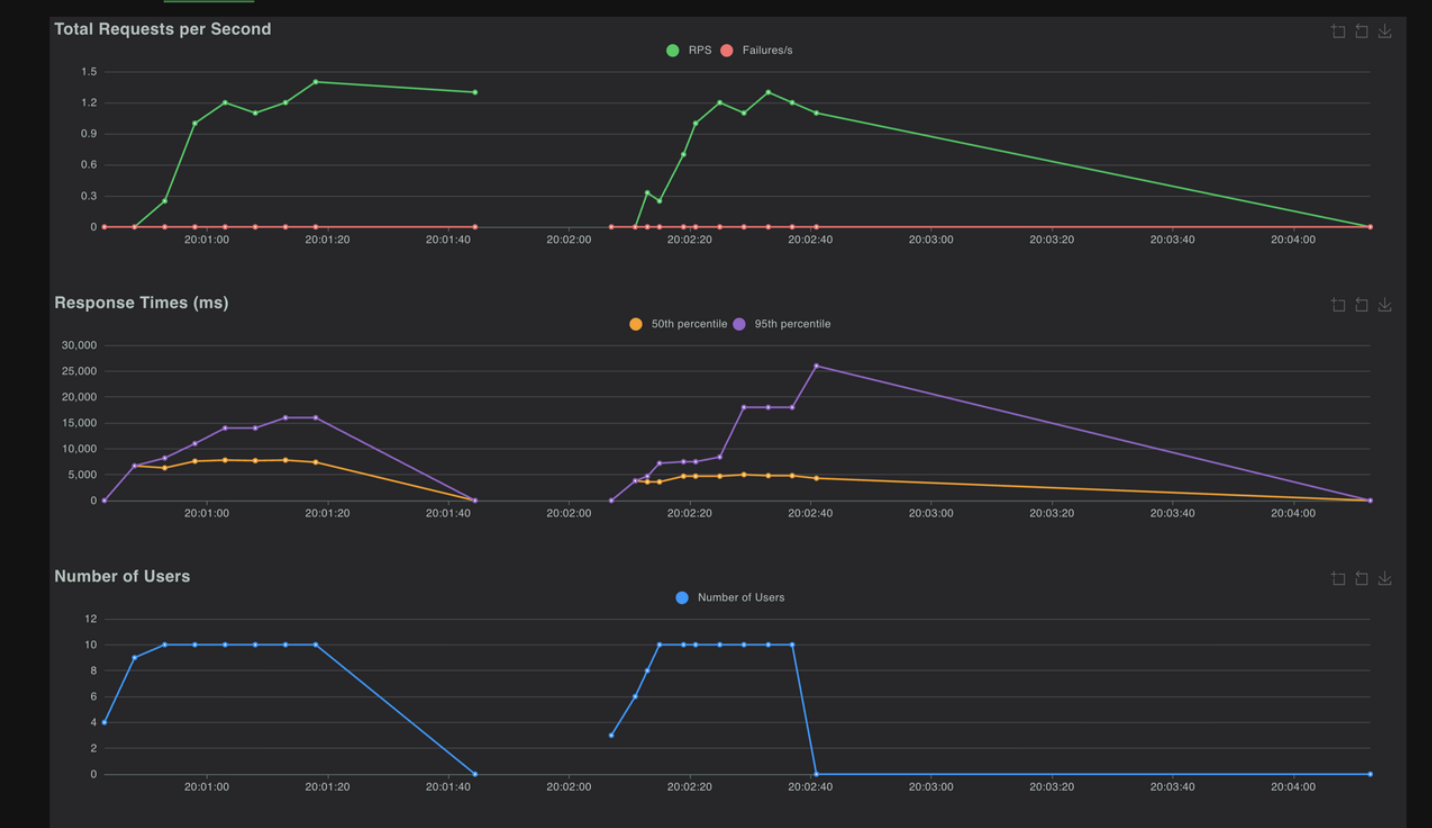
| 系统已经到最大吞吐量

多加用户只会排队。

这叫：Saturation Point（饱和点）

Replica = 8, users = 10

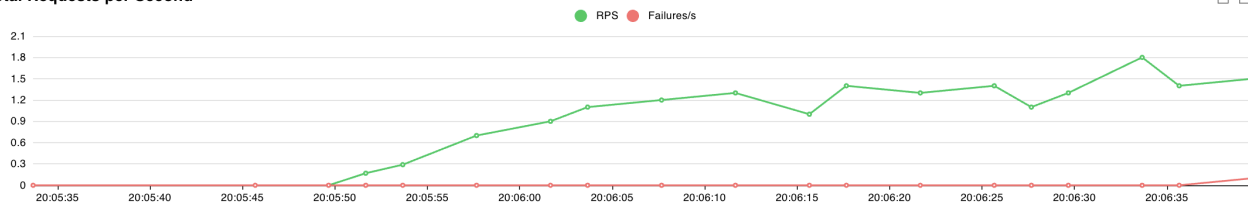
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/pose_estimation	35	0	4300	18000	26000	6473.4	2430	25723	1662.74	1.1	0
	Aggregated	35	0	4300	18000	26000	6473.4	2430	25723	1662.74	1.1	0



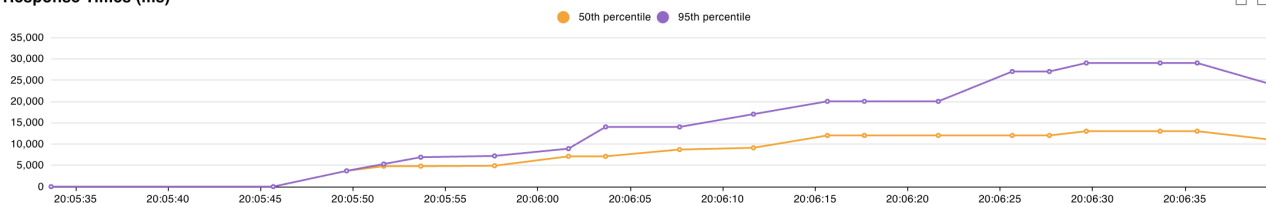
Replica = 8, users = 20

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/pose_estimation	63	1	11000	24000	29000	11517.67	3410	28644	1211.79	1.5	0.1
	Aggregated	63	1	11000	24000	29000	11517.67	3410	28644	1211.79	1.5	0.1

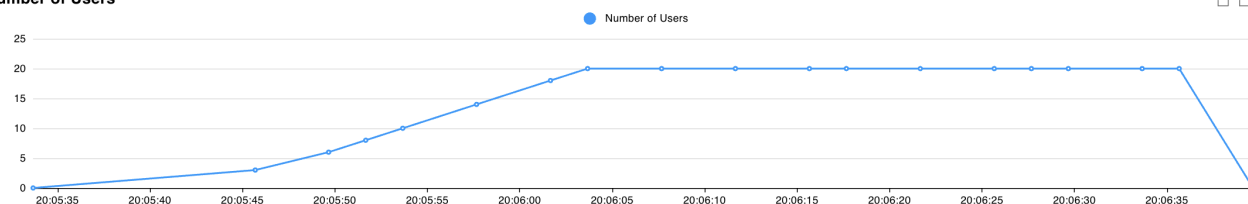
Total Requests per Second



Response Times (ms)



Number of Users



然后保持 Replica=8 但是 memory 调回 512mb

直接 OOMKill

```
ubuntu@k8s-master:~/25_fit5225_A1$ kubectl apply -f cloudpose-deployment.yaml -n jacky
deployment.apps/cloudpose-deployment configured
ubuntu@k8s-master:~/25_fit5225_A1$ kubectl rollout restart deployment cloudpose-deployment -n jacky
deployment.apps/cloudpose-deployment restarted
ubuntu@k8s-master:~/25_fit5225_A1$ kubectl get pod -n jacky -o wide
error: unable to match a printer suitable for the output format "wide", allowed formats are: custom-columns,custom-columns-file,go-template,go-template-file,json,jsonpath-as-json,jsonpath-file,name,template,templatefile,wide,yaml
ubuntu@k8s-master:~/25_fit5225_A1$ kubectl get pod -n jacky -o wide
NAME                                READY    STATUS    RESTARTS   AGE    IP             NODE           NOMINATED NODE   READINESS GATES
cloudpose-deployment-7474874958-2frcr 0/1      OOMKilled 1 (9s ago)  17s    10.47.0.2      k8s-worker2    <none>            <none>
cloudpose-deployment-7474874958-62dkj 0/1      OOMKilled 1 (10s ago) 17s    10.44.0.8      k8s-worker1    <none>            <none>
cloudpose-deployment-7474874958-6kml 0/1      OOMKilled 1 (11s ago) 19s    10.44.0.7      k8s-worker1    <none>            <none>
cloudpose-deployment-7474874958-9rc7k 0/1      OOMKilled 1 (11s ago) 19s    10.44.0.6      k8s-worker1    <none>            <none>
cloudpose-deployment-7474874958-c4x84 0/1      OOMKilled 1 (10s ago) 17s    10.47.0.1      k8s-worker2    <none>            <none>
cloudpose-deployment-7474874958-kqvwp 0/1      OOMKilled 1 (11s ago) 19s    10.47.0.7      k8s-worker2    <none>            <none>
cloudpose-deployment-7474874958-qdw7k 0/1      OOMKilled 1 (10s ago) 17s    10.44.0.9      k8s-worker1    <none>            <none>
cloudpose-deployment-7474874958-wh6km 0/1      OOMKilled 1 (11s ago) 19s    10.47.0.6      k8s-worker2    <none>            <none>
my-nginx-server-65b758d9bd-mm8rt      1/1      Running    0           2d5h   10.44.0.1      k8s-worker1    <none>            <none>
```

1 replica = 8, memory = 1Gi

→ 性能明显提升

→ RPS 上升

→ 延迟下降

→ 基本无失败

2 users 提高到 20

→ 系统再次进入饱和

→ 延迟上升

3 memory 改回 512Mi

→ 即使 replica = 8

→ 所有 Pod 直接 OOMKilled

这个结果说明一个非常重要的结论

你的 workload 是 Memory-Bound + CPU-Bound 双重特征

但 Memory 是硬门槛。

🎯 关键理解

横向扩展（replica 数量）只能解决：

- CPU 争用
- 并发排队

但无法解决：

- 单 Pod 内存不足

因为每个 Pod 都独立运行一份模型。

Horizontal scaling does not mitigate per-container memory constraints. When the memory limit was reduced to 512Mi, each replica independently exceeded its memory quota during model initialization, leading to immediate OOMKilled events, regardless of the number of replicas.

Scaling 类型差异

Scaling 类型	解决问题
Vertical scaling	单实例资源不足
Horizontal scaling	并发处理能力不足

你现在的 workload：

- 模型推理内存占用固定

- 并发请求吃 CPU

所以：

- 内存不足 → 必须 vertical scaling
- 并发过多 → horizontal scaling 有效

这就是云原生资源模型的核心思想。

看 replica=8 的性能：

你 RPS 达到 ≈ 1.5

对比：

- 1 副本 ≈ 0.2
- 2 副本 ≈ 0.6
- 8 副本 ≈ 1.5

不是线性 8 倍。

这说明：

- 节点 CPU 开始成为瓶颈
- 可能多个 Pod 被调度在同一 worker
- 或 CPU limit 500m 限制了吞吐

完整实验闭环

你验证了：

1. 单副本 → 低吞吐，高延迟
2. 双副本 → 明显改善
3. 高用户 → 饱和点出现
4. 高副本 → 吞吐持续提升但受节点限制
5. 降低内存 → 即使高副本仍 OOM

这就是：

Resource bottleneck identification experiment

下一步：使用 Azure cloud VM 开了一个 Windows instance，然后从这个 Azure 里发起对 OCI 的 Locust 压测请求。

**** 这部分本质就是作业的 Section 7 实验报告，已经全部做完，可以看这里：
https://github.com/Oatsinthepine/25_fit5225_A1/tree/main****

阶段背景

为了模拟真实生产环境中的 **外部客户端访问 Kubernetes 服务**，本实验采用以下架构：

- AI inference service 部署在 **OCI Kubernetes Cluster**
- 服务通过 **NodePort** 暴露
- 压测工具 **Locust** 运行在 **Azure VM (Windows)** 上
- Azure VM 作为 **远程负载生成器**

该架构模拟真实的 **跨云访问 + 外部客户端压测场景**。

为什么一定要从“另一台 VM”测？

这是个非常重要的设计点。

从 master 本地测：

- 没有公网网络延迟
- 可能绕过一些网络路径
- RTT 很低

从外部 VM 测：

- 有真实公网延迟

- 有 NAT / Security Group
- 有网络波动

老师想让你验证：

网络距离对分布式服务性能的影响

其 Azure cloud instance 的具体步骤请参考：[📄 Week02\(SUP\)-Introduction to Microsoft Azure.docx.pdf](#)

1 Azure VM 无法安装 WSL2

这是我一开始预想的结构：



Azure VM 使用 Windows image 时，尝试安装 WSL2：

在 PowerShell 中以 Admin 运行

```
1 wsl --install
2
3 PS C:\Windows\system32> wsl --install -d Ubuntu
4 Downloading: Ubuntu
5 Installing: Ubuntu
6 WSL2 is not supported with your current machine configuration.
7 Please enable the "Virtual Machine Platform" optional component and ensure
  virtualization is enabled in the BIOS.
8 Enable "Virtual Machine Platform" by running: wsl.exe --install --no-
  distribution
9 For information please visit https://aka.ms/enablevirtualization
10 Error code:
    Wsl/InstallDistro/Service/RegisterDistro/CreateVm/HCS/HCS_E_HYPERV_NOT_INSTALLE
    D
```

同时启用 Hyper-V 失败

代码块

```
1 # 先尝试启用 Hyper-V
2 dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all
  /norestart
3
4 # 在启用
5 dism.exe /online /enable-feature /featurename:Microsoft-Hyper-V-All /all
  /norestart
6
7 # 重启 VM
8 shutdown /r /t 0
9
10
11 ...
12 PS C:\Windows\system32> dism.exe /online /enable-feature
  /featurename:Microsoft-Hyper-V-All /all /norestart
13
14 Deployment Image Servicing and Management tool
15 Version: 10.0.26100.5074
16
17 Image Version: 10.0.26100.32370
18
19
20 Error: 0x800f080c
21
22 Feature name Microsoft-Hyper-V-All is unknown.
23 A Windows feature name was not recognized.
```

```
24 Use the /Get-Features option to find the name of the feature in the image and
    try the command again.
25
26 The DISM log file can be found at C:\Windows\Logs\DISM\dism.log
27
28 Feature name Microsoft-Hyper-V-All is unknown
```

Azure VM 没有启用 nested virtualization。

代码块

```
1 Azure物理机
2   ↓
3 Azure虚拟机 (你的Windows Server)
4   ↓
5 Hyper-V / WSL2
```

默认情况下，Azure 不允许在 VM 里面再开虚拟机。

而 WSL2 本质上就是一个 Hyper-V 虚拟机。

解决方案

放弃 WSL2，直接在 Windows 上安装：python + Git, 然后利用系统Python 装 venv，在 venv 中安装 locust

经验总结：

在云环境中：

VM ≠ 完整物理机

很多虚拟化功能：WSL2； Docker Desktop， Hyper-V 可能在云 VM 上无法正常使用。

2. Azure VM 无法直接使用powershell 脚本向 OCI instance 发命令（因为没有SSH private key）

问题

Azure VM 无法通过 SSH 连接 OCI master: Azure VM 这儿直接打 OCI 出现了 permission deny, 因为 pub key 的问题。我本地保存了 OCI master 节点的 pub key, 然后本地的 .ssh_config 保存了 private key, 但是 Azure VM 上没有这些。

代码块

```
1  Azure VM
2    ↓ SSH
3  OCI K8s master
```

但 **Azure VM 上没有你的 SSH private key**, 所以会出现 `Permission denied (publickey)` 因为 OCI 只允许我本地 Mac 的 private key 登录。

SSH 登录的机制是:

代码块

```
1  Server (OCI master)
2  保存的是 → public key
3
4  Client (Mac / Azure VM)
5  必须有 → private key
```

机器	有什么
Mac	private key ✓
OCI master	public key ✓
Azure VM	什么都没有 ✗

所以 Azure 无法登录

解决方案

把 private key 复制到 Azure VM 使用 SCP protocol。

只需要一次。

在 Mac 本地 terminal:

代码块

```
1  scp ~/.ssh/k8s-master-node.key jackyz@40.82.202.165:~/.ssh/
2
3  ** WARNING: connection is not using a post-quantum key exchange algorithm.
4  ** This session may be vulnerable to "store now, decrypt later" attacks.
5  ** The server may need to be upgraded. See https://openssh.com/pq.html
6  jackyz@40.82.202.165's password:
7  k8s-master-node.key                                100% 1671
   51.4KB/s   00:00
8
```

然后在 Azure VM 里设置权限：记得 `chmod 600`

测试 Azure VM 连接 OCI

记得这个命令要在 Git bash 里运行，不是 powershell

```
1  ssh -i ~/.ssh/k8s-master-node.key ubuntu@161.33.91.68
2
3  # 运行成功看到
4  ubuntu@k8s-master:~$
```

完整实验架构

代码块

```
1  Azure VM
2  |
3  | SSH
4  ▼
5  OCI K8s master
6  |
7  kubectl scale
8  |
9  Pods scaling
10 |
11 NodePort service
12 |
13 YOLO inference
```

```
14 |  
15 Locust load test  
16 |  
17 CSV metrics
```

然后我的实际实验架构：

代码块

```
1 Azure VM  
2 |  
3 └─ run_experiment.ps1  
4 |  
5 └─ ssh → scale deployment  
6 |  
7 └─ wait rollout  
8 |  
9 └─ run locust  
10 |  
11 └─ save csv
```

这样我之后就可以： `.\run_experiment.ps1`

跑完整个实验（包括 OCI pod 扩容 1/ 4/ 8， User 从 5 / 10 / 20）

PowerShell 在调用 ssh 时没有用到你刚配置的 ~/.ssh/config 或 key 路径。你目前已经能手动 ssh oci-master 成功，说明 SSH config 本身是正确的，问题只在脚本里。

方案：脚本里直接指定 key

代码块

```
1 ssh -i $HOME/.ssh/id_rsa ubuntu@161.33.91.68 "kubectl scale deployment  
cloudpose-deployment --replicas=$pods -n jacky"  
2  
3 ssh -i $HOME/.ssh/id_rsa ubuntu@161.33.91.68 "kubectl rollout status  
deployment cloudpose-deployment -n jacky"
```

这样就不会依赖 config

实际上我最后完整的 powershell 自动测试脚本:

代码块

```
1  # CloudPose Benchmark Experiment Runner
2  #
3  # Runs automated benchmarking experiments.
4  #
5  # Steps:
6  # 1) Scale Kubernetes deployment on OCI cluster
7  # 2) Wait for rollout
8  # 3) Run Locust load test from Azure VM
9  # 4) Save CSV results to experiment/results
10 #
11 # Requirements:
12 # - Windows PowerShell
13 # - Locust installed
14 # - SSH key configured for OCI master node
15 #
16 # Usage:
17 # Run from repository root:
18 # .\experiment\run_experiment.ps1
19
20 cd C:\locust_test\25_fit5225_A1
21
22 $HOST_URL = "http://161.33.91.68:30007"
23 $LOCUSTFILE = "client\locustfile.py"
24 $RESULT_DIR = "experiment\results"
25
26 Write-Host "===== "
27 Write-Host " - CloudPose Benchmark Experiment Start - "
28 Write-Host "===== "
29
30 $experiments = @(
31     @{users=5; pods=1},
32     @{users=10; pods=1},
33     @{users=20; pods=1},
34
35     @{users=20; pods=4},
36     @{users=40; pods=4},
37
38     @{users=40; pods=8}
39 )
40
```

```
41  foreach ($exp in $experiments) {
42
43      $users = $exp.users
44      $pods = $exp.pods
45
46      Write-Host ""
47      Write-Host "-----"
48      Write-Host "Running experiment:"
49      Write-Host "Users = $users"
50      Write-Host "Pods = $pods"
51      Write-Host "-----"
52
53
54      # Step 1 change replicas
55      Write-Host "Scaling Kubernetes deployment..."
56      ssh -i $HOME/.ssh/k8s-master-node.key ubuntu@161.33.91.68 "kubectl scale
deployment cloudpose-deployment --replicas=$pods -n jacky"
57
58      # Step 2 wait for rollout of the pods and update then ready
59      Write-Host "Waiting for rollout to complete..."
60      ssh -i $HOME/.ssh/k8s-master-node.key ubuntu@161.33.91.68 "kubectl rollout
status deployment cloudpose-deployment -n jacky"
61
62
63      Write-Host "Pods are ready."
64
65      # Run locust test
66      Write-Host "Starting Locust load test..."
67
68      # Step 3 run locust
69      locust `
70          -f $LOCUSTFILE `
71          --host $HOST_URL `
72          --headless `
73          -u $users `
74          -r 2 `
75          --run-time 2m `
76          --csv "$RESULT_DIR\exp_${users}u_${pods}p"
77
78      Write-Host "Experiment completed."
79  }
80
81  Write-Host ""
82  Write-Host "====="
83  Write-Host " All experiments finished successfully"
84  Write-Host " Results saved in experiment/results/"
85  Write-Host "====="
```